Documentation du projet KhedMAP

Guide simple pour comprendre, lancer, modifier et continuer le projet apres nous.

Version: 05/05/2026 - Projet organise en frontend et backend

Objectif de cette documentation

Cette documentation est faite pour une personne qui recupere le projet sans connaitre le code. Elle explique le role de chaque dossier, comment demarrer l'application, quelles fonctionnalites existent, et comment ajouter une nouvelle page, une nouvelle route ou une nouvelle fonctionnalite sans casser le reste.

1. Idee generale du projet

KhedMAP est une application de mise en relation entre clients et freelances. Un client peut publier un besoin, consulter des services, contacter un freelance et commander. Un freelance peut creer son profil, publier ses services, postuler a des offres et suivre ses commandes.

Le projet contient une interface moderne en Ionic/Angular et un backend Flask qui expose une API connectee a MongoDB. L'idee importante a retenir: le frontend affiche les pages et appelle l'API; le backend verifie, enregistre et renvoie les donnees.

2. Structure propre du projet

```
KhedMAP/
|-- frontend/              Interface Ionic + Angular
|   |-- src/
|   |   |-- app/
|   |   |   |-- pages/     Pages de l'application
|   |   |   |-- services/  Appels API et logique cote frontend
|   |   |   |-- models/    Types et modeles TypeScript
|   |   |-- assets/        Images et icones
|   |   |-- environments/  Configuration API dev/prod
|   |-- package.json       Dependances frontend et scripts Angular
|   |-- angular.json       Configuration Angular
|   |-- proxy.conf.js      Redirection /api vers Flask
|
|-- backend/               API Flask + MongoDB
|   |-- app/
```

```
|   |   |-- routes/        Routes API et routes web
|   |   |-- repositories/  Acces a MongoDB
|   |   |-- services/      Logique metier et dashboards
|   |   |-- utils/         Fonctions utiles: auth, formatage, serialisation
|   |-- templates/         Pages HTML backend
|   |-- static/            CSS backend
|   |-- requirements.txt   Dependances Python
|   |-- run.py             Point de lancement Flask
|
|-- documentation.pdf      Ce guide
|-- package.json           Scripts pratiques depuis la racine
|-- .gitignore             Fichiers a ne pas pousser sur GitHub
```

Pourquoi cette structure est meilleure ?

Avant, le frontend et les fichiers racine Angular etaient melanges avec le backend. Maintenant, chaque partie a sa place: un developpeur frontend travaille dans `frontend/`, un developpeur backend travaille dans `backend/`. Le projet est plus facile a lire, a presenter et a maintenir.

3. Technologies utilisees

Frontend

- **Ionic**: composants mobiles et responsive.
- **Angular**: structure de l'application.
- **TypeScript**: code plus clair avec des types.
- **SCSS**: styles des pages.

Backend

- **Flask**: serveur web et API.
- **MongoDB**: base de donnees NoSQL.
- **PyMongo**: communication Python avec MongoDB.
- **Flask-CORS**: autorise le frontend a appeler l'API.

4. Installation sur un nouveau PC

4.1 Prerequis

- Installer Git.
- Installer Node.js et npm.
- Installer Python 3.
- Installer MongoDB ou avoir une URL MongoDB distante.

4.2 Recuperer le projet depuis GitHub

```
git clone https://github.com/AbirSaafi02/KhedMap.git
cd KhedMap
```

Pour recuperer exactement la version documentee ici:

```
git checkout f75ad98
```

4.3 Installer le frontend

```
cd frontend
npm install
cd ..
```

4.4 Installer le backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
cd ..
```

Sur macOS/Linux, utiliser `source .venv/bin/activate` et `cp .env.example .env`.

5. Configuration importante

Le fichier `backend/.env` contient la configuration locale. Il ne doit pas etre pousse sur GitHub, parce qu'il peut contenir des secrets ou des informations propres a la machine.

```
SECRET_KEY=dev-khedmap-secret
MONGO_URI=mongodb://127.0.0.1:27017/
MONGO_DB_NAME=khedmap
AUTO_SEED=true
BACKEND_HOST=127.0.0.1
BACKEND_PORT=5000
FRONTEND_HOST=127.0.0.1
FRONTEND_PORT=8100
```

Si MongoDB tourne en local, garder `MONGO_URI=mongodb://127.0.0.1:27017/`. Si la base est distante, remplacer cette valeur par l'URL fournie.

6. Comment lancer l'application

Depuis la racine `KhedMAP/`, ouvrir deux terminaux.

Terminal 1: backend

```
npm run backend
```

Terminal 2: frontend

```
npm run frontend
```

Adresses par defaut:

- Application: `http://127.0.0.1:8100`
- Backend: `http://127.0.0.1:5000`
- Test API: `http://127.0.0.1:5000/api/health`

> **Comment le frontend parle au backend ?**
>
> Le frontend utilise `/api`. En developpement, `frontend/proxy.conf.js` redirige automatiquement ces appels vers Flask sur le port 5000. Cela evite d'ecrire l'URL complete dans chaque service.

7. Fonctionnalites principales

Authentification

- Creation de compte avec nom, email, mot de passe et role.
- Connexion et deconnexion.
- Stockage de l'utilisateur connecte cote frontend.
- Protection des routes API avec session.

Parcours client

- Acceder a la page d'accueil client.
- Modifier son profil.
- Publier une offre de travail.
- Voir les offres publiees.
- Consulter les gigs et details d'un gig.
- Consulter un profil freelance.
- Commander un gig ou un produit.
- Envoyer et recevoir des messages.

Parcours freelance

- Acceder a la page d'accueil freelance.
- Modifier son profil.
- Voir les offres disponibles.
- Postuler a une offre.
- Creer et gerer ses gigs.
- Creer des produits/services vendables.
- Suivre les commandes recues.
- Envoyer et recevoir des messages.

Fonctionnalites partagees

- Store pour consulter les produits.
- Chat et conversations.
- Notifications.
- Dashboard selon le role connecte.

Administration

Une partie administration existe aussi, mais elle n'est pas le premier parcours utilisateur. Elle sert surtout a suivre la plateforme, voir une vue globale et valider certains elements comme les comptes, gigs, produits ou signalements.

8. Ou se trouvent les pages frontend ?

Besoin	Dossier
Pages client	`frontend/src/app/pages/client/`
Pages freelance	`frontend/src/app/pages/freelancer/`
Login et inscription	`frontend/src/app/pages/login/` et `frontend/src/app/pages/register/`
Store	`frontend/src/app/pages/store/`
Chat	`frontend/src/app/pages/chat/`
Notifications	`frontend/src/app/pages/notifications/`
Routes Angular	`frontend/src/app/app.routes.ts`

9. Ou se trouvent les services frontend ?

Les services sont importants parce qu'ils evitent de mettre les appels API directement dans les pages. Une page appelle un service, et le service appelle l'API.

Service	Role
`api.ts`	Service HTTP central. Tous les appels passent par lui.
`auth.ts`	Connexion, inscription, session et utilisateur courant.
`job.service.ts`	Offres, details d'une offre, creation et candidature.
`marketplace.service.ts`	Gigs, produits, commandes liees au marketplace.
`conversation.service.ts`	Conversations et messages.
`dashboard.service.ts`	Donnees des dashboards.
`notification.service.ts`	Notifications utilisateur.
`order.service.ts`	Commandes client/freelance.

10. Organisation du backend

Le backend est decoupe en couches pour garder le code lisible.

Dossier/Fichier	Role
`backend/run.py`	Lance l'application Flask.
`backend/app/__init__.py`	Cree et configure l'application Flask.
`backend/app/config.py`	Lit les variables d'environnement et configure MongoDB, ports, CORS.
`backend/app/routes/api.py`	Contient les routes JSON appelees par Angular.
`backend/app/routes/web.py`	Routes HTML backend avec templates Jinja.
`backend/app/repositories/`	Fonctions qui lisent et modifient MongoDB.
`backend/app/services/`	Logique plus globale: dashboards, seed, emails.
`backend/app/utils/`	Petites fonctions reutilisables.

11. Routes API importantes

Route	Utilite
`GET /api/health`	Verifier que le backend et MongoDB repondent.
`POST /api/auth/register`	Creer un compte.
`POST /api/auth/login`	Se connecter.
`POST /api/auth/logout`	Se deconnecter.
`GET /api/me`	Recuperer l'utilisateur connecte.
`PATCH /api/me`	Modifier le profil.
`GET /api/jobs`	Lister les offres.
`POST /api/jobs`	Creer une offre client.
`POST /api/jobs/<id>/apply`	Postuler a une offre.
`GET /api/gigs`	Lister les gigs.
`POST /api/gigs`	Creer un gig freelance.
`POST /api/gigs/<id>/order`	Commander un gig.
`GET /api/products`	Lister les produits.
`POST /api/products`	Creer un produit.
`POST /api/products/<id>/buy`	Acheter un produit.
`GET /api/conversations`	Lister les conversations.
`POST /api/conversations/<id>/messages`	Envoyer un message.
`GET /api/notifications`	Lister les notifications.

12. Base de donnees

La base par defaut s'appelle `khedmap`.

Collections principales:

- `users` : utilisateurs et roles.

- `jobs` : offres publiees par les clients.
- `applications` : candidatures des freelances.
- `gigs` : services proposes par les freelances.
- `products` : produits/services vendables.
- `orders` : commandes.
- `conversations` : discussions et messages.
- `notifications` : notifications utilisateur.
- `reports` : signalements.

Si `AUTO_SEED=true` , le backend peut ajouter des donnees de demonstration quand la base est vide.

13. Comment ajouter une nouvelle page frontend

1. Creer un dossier dans `frontend/src/app/pages/` .
2. Ajouter les fichiers de page: `.ts` , `.html` , `.scss` .
3. Ajouter la route dans `frontend/src/app/app.routes.ts` .
4. Si la page appelle le backend, creer ou reutiliser un service dans `frontend/src/app/services/` .
5. Tester dans le navigateur avec `npm run frontend` .

Exemple de route:

```
{ path: 'client/example', loadComponent: () =>
  import('./pages/client/example/example.page').then(m => m.ExamplePage) }
```

14. Comment ajouter une nouvelle route backend

1. Ajouter la route dans `backend/app/routes/api.py` .
2. Si la route touche MongoDB, ajouter une fonction dans le repository correspondant.
3. Verifier les droits avec `@login_required` ou `@role_required(...)` .
4. Retourner une reponse JSON simple, par exemple `{"data": ...}` .
5. Tester avec le frontend ou avec l'URL API.

Exemple:

```
@api_bp.get("/example")
@login_required
def api_example():
    return jsonify({"data": {"message": "ok"}})
```

15. Comment ajouter une nouvelle collection MongoDB

1. Creer un fichier dans `backend/app/repositories/`.
2. Mettre dedans les fonctions `create`, `find`, `list`, `update` selon le besoin.
3. Utiliser ces fonctions dans les routes, pas directement la base dans la page frontend.
4. Ajouter des donnees demo dans `backend/app/services/seed.py` si c'est utile.

16. Comment verifier que tout fonctionne

Build frontend

```
npm run frontend:build
```

Si le build passe, Angular arrive a compiler l'application. Des warnings de budget SCSS peuvent apparaitre: ce ne sont pas des erreurs bloquantes.

Backend

```
npm run backend
```

Puis ouvrir:

```
http://127.0.0.1:5000/api/health
```

Verification manuelle

- Ouvrir `http://127.0.0.1:8100`.
- Tester inscription/connexion.
- Tester une page client.
- Tester une page freelance.
- Verifier que les appels API ne donnent pas d'erreur dans la console.

17. Ce qu'il ne faut pas pousser sur GitHub

Ces fichiers sont generes localement ou contiennent des informations propres a la machine:

- `frontend/node_modules/`
- `frontend/www/`
- `frontend/.angular/`
- `.venv/`

- `backend/.env`
- `__pycache__/`

Le fichier `.gitignore` est la pour eviter de les envoyer par erreur.

18. Guide rapide pour reprendre le projet

1. Lire cette documentation.
2. Lancer le backend et le frontend.
3. Commencer par regarder les routes dans `frontend/src/app/app.routes.ts`.
4. Lire les services dans `frontend/src/app/services/` pour comprendre les appels API.
5. Lire `backend/app/routes/api.py` pour comprendre ce que le backend expose.
6. Lire les repositories pour comprendre comment MongoDB est utilisee.
7. Modifier une seule fonctionnalite a la fois et tester apres chaque changement.

Resume final

KhedMAP est maintenant organise de maniere claire: le frontend est separe du backend, le PDF explique comment reprendre le projet, et GitHub contient uniquement les fichiers necessaires. Une nouvelle personne peut cloner le projet, installer les dependances, configurer MongoDB et commencer a travailler sans deviner la structure.